

Sai Kiran Belana

saikiransanju22@gmail.com | +1-959-995-3083

 belanasaikiran.github.io |  linkedin.com/in/belanasaikiran/ |  github.com/belanasaikiran

EDUCATION

University of Connecticut

Masters in Computer Science Engineering

Expected Graduation: December 2025

GPA: 3.6/4.0

Coursework: Operating Systems, Computer Architecture, Computer Security, Cyber Security, Algorithms, Computer Networks

PROFESSIONAL EXPERIENCE

Student Web Developer, Campus Technology Services, University of Connecticut

February 2024 - Present

- Created Docker configurations to transition university applications to OpenShift Cloud Containers, **reducing resource utilization by 60%** and enabling seamless autoscaling .
- Consolidated five classroom platforms into a single system with advanced filtering, **reducing maintenance costs by 80%**.
- Designed and deployed the OneDrive restoration application using PHP and Bitbucket pipelines.
- Developed custom WordPress plugins for block-based components, integrating custom query ACF blocks to support multiple filters for renderable layouts.

Software Engineer, LightSpeed Photonics

April 2021 - December 2023

- Automated Petalinux (Yocto) OS image generation for Xilinx FPGA boards using Bash scripting which reduced **build time from 4 hours to 1 hour**.
- Architected a RESTful web platform integrated with Slurm Workload Manager, enabling remote FPGA application deployment on HPC clusters — **reducing manual deployment effort by over 85%**.
- Distributed tile based matrix multiplication computation across multiple FPGAs, **benchmarking OpenCL kernel performance b/w Xeon CPUs and FPGAs**.
- Managed cloud resources across AWS, Azure, and GCP, handling networking, EC2 instances, S3 storage, and K8s deployment.

PROJECTS

WayNotify - A notification daemon for Wayland

[C++, CMake, Wayland, Cairo]

- Building a background service that listens for desktop notifications (via D-Bus), then displays them on-screen using Cairo and wlroots. This serves as a drop-in replacement for dunst.

System Calls Implementation and Testing on OS161

[Operating Systems, C, Linux Kernel, GCC, Makefile]

- Implemented synchronization primitives (locks and semaphores) in OS/161 to coordinate execution across threads and processes, preventing deadlocks and resource starvation.
- Modified and recompiled Linux Kernel tree (v5.14) to add custom kernel modules and validated output logs using dmesg.

CPU Scheduling & File System Management Simulation

[Operating Systems, C++, Linux, Dear ImGui]

- Successfully simulated job execution process of CPU scheduling algorithms(FCFS, SJF, SRTF, RR) using Dear ImGui library.
- Built an interactive Linux app that invoked system calls to read/write files, monitor disk usage, modify files & set permissions.

RISC-V Cache Architecture

[C, GCC, Assembly]

- Constructed an out-of-order execution pipeline which supports 1-, 2-, and 4-way set associative caches.
- Implemented 1-level and 2-level dynamic branch predictors (2-bit) using BHT and BHSR tables.
- Compared performance metrics by testing set associativity and branch predictors to identify the best configuration.

Buffer Overflow Exploit & Mitigation

[C, Assembly, Shellcode Injection, xxd]

- Gained root privileges by injecting shellcode and manipulating return addresses on 32/64-bit systems.
- Evaluated defenses like StackGuard, ASLR, and non-executable stacks through practical testing and analysis.

Dirty COW Attack

[C, mmap]

- Demonstrated a privilege escalation attack by exploiting a race condition using the Dirty Copy-On-Write vulnerability in a controlled Ubuntu 12.04 environment.

TCP/IP Socket Programming and Network Protocol Implementation

[Python, OpenFlow, Mininet, iperf]

- Analyzed throughput and latency under varying workloads by developing and evaluating single-threaded and multithreaded TCP socket client-server systems handling up to 2 concurrent clients

Car Make & Model Detection using Xception Model

[Python, Keras, TensorFlow, Xception, Stanford Car Dataset]

- Built an ML model using Xception to classify car make and model for the Stanford Car Dataset (196 classes, 16,185 images).
- Achieved 99.69% accuracy & 0.58% loss on test data and demonstrated its use-case for real-time traffic monitoring.

SKILLS

Programming Languages: C, C++, SQL, Shell Scripting, JavaScript, PHP, Dart, Java

Frameworks: OpenCL, ZeroMQ, RestAPI, Flutter, Electron, NodeJs, ReactJs, SQL, MongoDB

Software Tools & Cloud : GDB, Git, Docker, Kubernetes, Slurm Workload Manager, Vagrant, Azure, AWS, GCP

Embedded Devices: Raspberry Pi, Linux, Arduino, ESP32, Microcontrollers & Sensors, Yocto Project, RTOS